

Univerzitet u Sarajevu
Elektrotehnički Fakultet
Objektno Orijentisana Analiza i Dizajn

ETF Talent Program

Deseti projektni zadatak
Paterni ponašanja

Maj 2026.

Sadržaj

1	Paterni ponašanja — opći pregled	2
1.1	1. Chain of Responsibility	2
1.2	2. Command	2
1.3	3. Iterator	3
1.4	4. Mediator	3
1.5	5. Memento	3
1.6	6. Observer	4
1.7	7. State	4
1.8	8. Strategy	5
1.9	9. Template Method	5
1.10	10. Visitor	6
2	Odabir paterna za ETF Talent Program	6
3	Primjena u sistemu ETF Talent Program	7
3.1	Observer patern — Sistem notifikacija	7
3.1.1	Problem bez paterna	7
3.1.2	Rješenje primjenom Observer paterna	7
3.2	Strategy patern — Algoritam rangiranja studenta	8
3.2.1	Problem bez paterna	8
3.2.2	Rješenje primjenom Strategy paterna	8
4	Pregled dodanih klasa	9

1 Paterni ponašanja — opći pregled

Paterni ponašanja (*Behavioral Design Patterns*) bave se algoritmima i raspodjelom odgovornosti između objekata. Za razliku od kreacionih paterna koji opisuju *kako* se objekti kreiraju, ili strukturalnih paterna koji opisuju *kako* se klase komponuju u veće cjeline, paterni ponašanja definišu *kako objekti međusobno komuniciraju* i kako se odgovornost za izvršavanje određene akcije dijeli između više objekata u sistemu.

Postoji deset klasičnih paterna ponašanja, definisanih u knjizi *Design Patterns: Elements of Reusable Object-Oriented Software* (Gang of Four, 1994).

1.1 1. Chain of Responsibility

Svrha: Prosljeđuje zahtjev duž lanca handlera. Svaki handler odlučuje da li će obraditi zahtjev ili ga proslijediti sljedećem handleru u lancu.

Struktura: Apstraktni handler definira metodu `handle(request)` i čuva referencu na sljedećeg handlera. Konkretni handleri implementiraju logiku obrade ili prosljeđivanja. Klijent šalje zahtjev prvom handleru, ne znajući koji će ga obraditi.

Kada se koristi:

- Kada više od jednog objekta može obraditi zahtjev, a handler nije unaprijed poznat
- Kada se želi proslijediti zahtjev jednom od više handlera bez eksplicitnog specificiranja koji handler treba da ga obradi
- Primjeri: middleware pipeline u web aplikacijama, validacija forme kroz više koraka, autorizacijski lanac provjera

Primjer iz prakse: HTTP zahtjev prolazi kroz lanac middleware komponenti (autentifikacija → autorizacija → logovanje → obrada) pri čemu svaka komponenta može prekinuti lanac ili ga nastaviti.

1.2 2. Command

Svrha: Enkapsulira zahtjev kao objekt, čime se omogućava parametrizacija klijenata s različitim zahtjevima, stavljanje zahtjeva u red čekanja, logovanje zahtjeva i podrška za operacije koje se mogu poništiti (*undo*).

Struktura: Interfejs `ICommand` definira metodu `execute()`. Konkretni komande implementiraju interfejs i čuvaju sve potrebne parametre za izvršavanje. Invoker pokreće komandu, a Receiver je objekat koji zapravo izvršava operaciju.

Kada se koristi:

- Kada se želi parametrizirati akcija koja se treba izvršiti
- Kada se želi podržati undo/redo funkcionalnost
- Kada se želi implementirati transakcijski sistem ili red čekanja akcija
- Primjeri: akcije u GUI editorima, transakcije u bazi podataka, taskovi u task scheduleru

Primjer iz prakse: U tekst editoru, svaka radnja (pisanje, brisanje, formatiranje) je implementirana kao komanda objekt koji podržava `execute()` i `undo()`.

1.3 3. Iterator

Svrha: Pruža način sekvencijalnog pristupa elementima kolekcije bez izlaganja njene unutrašnje reprezentacije. Odvaja logiku obilaska kolekcije od same kolekcije.

Struktura: Interfejs `IEnumerator` definira metode `hasNext()` i `next()`. Konkretni iterator implementira logiku obilaska. Kolekcija nudi metodu `createIterator()` koja vraća odgovarajući iterator. Klijent koristi iterator bez poznavanja internih detalja kolekcije.

Kada se koristi:

- Kada se želi uniforman način pristupa različitim tipovima kolekcija
- Kada se želi podržati više načina obilaska iste kolekcije (npr. sortirani, filtrirani, reverzni)
- Primjeri: obilazak stabla, paginacija rezultata pretrage, streaming podataka

Primjer iz prakse: LINQ u C# i `foreach` petlja implementiraju Iterator patern — klijentski kod ne zna da li iterira po listi, skupu ili bazi podataka.

1.4 4. Mediator

Svrha: Definira objekt koji enkapsulira način na koji skup objekata međusobno interaguje. Smanjuje direktne zavisnosti između objekata tako što ih primorava da komuniciraju isključivo kroz centralni medijator objekt.

Struktura: Interfejs medijatora definira metode za komunikaciju. Konkretni medijator implementira koordinacijsku logiku i čuva reference na sve kolege objekte. Kolega objekti znaju samo za medijatora, a ne jedni za druge.

Kada se koristi:

- Kada velik broj klasa međusobno direktno komunicira, uzrokujući teško razumljivu mrežu zavisnosti
- Kada se želi centralizirati kompleksnu koordinacijsku logiku
- Primjeri: chat server koji koordinira poruke između korisnika, UI forme gdje promjena jednog polja utiče na stanje više drugih polja

Primjer iz prakse: U chat aplikaciji, korisnici ne šalju poruke direktno jedni drugima — sve prolazi kroz chat room (medijator) koji ih distribuira.

1.5 5. Memento

Svrha: Bez narušavanja enkapsulacije, hvata i eksternalizira unutrašnje stanje objekta kako bi se isti objekt mogao kasnije vratiti u to stanje (*undo*).

Struktura: Originator kreira memento objekt koji sadrži snimak unutrašnjeg stanja i može obnoviti to stanje iz memento. Caretaker čuva memento ali nikad ne pristupa njegovom sadržaju — samo ga čuva i vraća originatoru kada zatreba.

Kada se koristi:

- Kada je potrebno implementirati undo/redo funkcionalnost
- Kada direktno izlaganje unutrašnjeg stanja objekta bi narušilo enkapsulaciju
- Primjeri: history u tekst editorima, checkpointi u igricama, verzionisanje konfiguracije

Primjer iz prakse: Ctrl+Z u tekst editoru — editor čuva memento snimke stanja dokumenta i vraća ih na zahtjev korisnika.

1.6 6. Observer

Svrha: Definiira zavisnost jedan-na-više između objekata, tako da kada jedan objekt promijeni stanje, svi zavisni objekti automatski dobivaju obavijest i ažuriraju se. Poznat i kao *Publish-Subscribe* patern.

Struktura: Subject (Publisher) čuva listu observera i nudi metode `subscribe()`, `unsubscribe()` i `notify()`. Interfejs observera definiira metodu `update()`. Konkretni observeri implementiraju reakciju na promjenu. Subject ne zna ništa o konkretnim implementacijama observera — ovisi samo o apstraktnom interfejsu.

Kada se koristi:

- Kada promjena stanja jednog objekta zahtijeva promjenu u drugima, a nije unaprijed poznato koliko objekata se mora promijeniti
- Kada se želi slabo sprežanje između objekata koji trebaju međusobno komunicirati
- Primjeri: event sistemi u GUI frameworkcima, notifikacijski sistemi, reaktivno programiranje

Primjer iz prakse: Newsletter sistem — pretplatnici se prijavljuju i odjavljuju, a publisher ih automatski obavještava o novim objavljenim člancima bez poznavanja konkretnih pretplatnika.

1.7 7. State

Svrha: Omogućava objektu da promijeni svoje ponašanje kada se promijeni njegovo unutrašnje stanje. Objekt djeluje kao da je promijenio svoju klasu.

Struktura: Context objekt čuva referencu na trenutno stanje i delegira sve zahtjeve njemu. Apstraktno stanje definiira interfejs za sva konkretna stanja.

Konkretna stanja implementiraju specifično ponašanje i mogu mijenjati stanje Context objekta.

Kada se koristi:

- Kada ponašanje objekta ovisi o njegovom stanju i mora se mijenjati u runtime-u
- Kada kod sadrži veliki broj `if/else` ili `switch` iskaza koji provjeravaju stanje objekta
- Primjeri: order lifecycle (kreiran → plaćen → isporučen), korisnički račun (aktivan → suspendovan → obrisani), vending mašina

Primjer iz prakse: Oglas u sistemu može biti u stanju AKTIVAN, ARHIVIRAN ili OBRISAN — svako stanje nudi drugačije ponašanje za iste operacije (npr. prijava na oglas je moguća samo u stanju AKTIVAN).

1.8 8. Strategy

Svrha: Definira porodicu algoritama, enkapsulira svaki od njih i čini ih međusobno zamjenjivim. Strategy patern omogućava promjenu algoritma neovisno od klijenata koji ga koriste.

Struktura: Interfejs strategije definira ugovor koji implementiraju sve konkretne strategije. Context objekt čuva referencu na trenutnu strategiju i delegira izvršavanje njoj. Klijent odabire i postavlja konkretnu strategiju u Context. Context ne zna ništa o internoj implementaciji strategije.

Kada se koristi:

- Kada postoji više varijanti istog algoritma koje treba moći mijenjati u runtime-u
- Kada se želi eliminisati veliki broj `if/else` iskaza koji biraju između varijanti algoritma
- Kada se želi izolirati implementacijske detalje algoritma od koda koji ga koristi
- Primjeri: različite strategije sortiranja, načini plaćanja u e-commerce aplikaciji, strategije kompresije podataka

Primjer iz prakse: Navigacijska aplikacija nudi više strategija rute: najbrža, najkraća, izbjegavanje autoputeva. Korisnik bira strategiju, a aplikacija koristi odabranu bez mijenjanja ostatka koda.

1.9 9. Template Method

Svrha: Definira kostur algoritma u baznoj klasi, ostavljajući određene korake podklasama da ih implementiraju. Template Method omogućava podklasama da redefinišu određene korake algoritma bez mijenjanja njegove ukupne strukture.

Struktura: Apstraktna bazna klasa sadrži template metodu koja definira slijed koraka. Neki koraci su implementirani u baznoj klasi (invarijantni), a

drugi su apstraktni ili virtualni (varijantni). Podklase nasljeđuju baznu klasu i implementiraju varijantne korake.

Kada se koristi:

- Kada više klasa dijeli isti algoritamski kostur, ali se razlikuju u nekim koracima
- Kada se želi kontrolisati tačke proširenja algoritma
- Primjeri: parsiranje različitih formata fajlova, generisanje različitih tipova izvještaja, životni ciklus web zahtjeva

Primjer iz prakse: Proces generisanja izvještaja: dohvati podatke → filtriraj → formatiraj → exportuj. Svaki tip izvještaja (PDF, Excel, CSV) implementira samo korake koji su specifični za njega.

1.10 10. Visitor

Svrha: Omogućava definisanje novih operacija nad elementima objektno strukture bez mijenjanja klasa tih elemenata. Odvaja algoritam od strukture objekata na kojima operira.

Struktura: Interfejs visitora definira metodu `visit()` za svaki tip elementa. Konkretni visitor implementira specifičnu operaciju za svaki tip. Elementi implementiraju metodu `accept(visitor)` koja poziva `visitor.visit(this)`. Double dispatch osigurava poziv ispravne metode na osnovu tipa i visitora i elementa.

Kada se koristi:

- Kada se želi dodati nova operacija nad heterogenom strukturom objekata bez mijenjanja tih klasa
- Kada ista operacija mora biti primijenjena na objekte različitih tipova
- Primjeri: generisanje koda iz AST stabla, export podataka u različite formate, validacija složene strukture dokumenata

Primjer iz prakse: Kompajler obilazi stablo apstraktne sintakse (AST) gdje svaki čvor ima `accept(visitor)`. Različiti visitori vrše generisanje koda, optimizaciju ili analizu tipova.

2 Odabir paterna za ETF Talent Program

Od navedenih deset paterna ponašanja, za sistem ETF Talent Program odabrana su dva: **Observer** i **Strategy**.

Observer je odabran jer sistem posjeduje klasu `EmailNotifikacija` koja mora reagovati na događaje koji se dešavaju u više dijelova sistema istovremeno — prijava studenta na oglas, slanje ponude, odobrenje verifikacije. Trenutno, logika slanja notifikacija je raspoređena po cijelom kodu kontrolera. Observer patern centralizira ovu logiku i eliminira direktnu zavisnost između poslovnih klasa i mehanizma notifikacija.

Strategy je odabran jer algoritam rangiranja studenta (klasa `RangParametri`) sadrži fiksnu formulu koja ne može biti lako zamijenjena. Kako sistem evoluira,

mogu se pojaviti različite strategije rangiranja — po akademskom uspjehu, po projektu, po kombinaciji faktora ili po specifičnoj oblasti. Strategy patern omogućava dodavanje novih strategija rangiranja bez izmjene klase `StudentProfil`.

3 Primjena u sistemu ETF Talent Program

3.1 Observer patern — Sistem notifikacija

3.1.1 Problem bez paterna

Trenutno u sistemu, kada se dogodi neka ključna akcija (prijava studenta, slanje ponude, odobrenje verifikacije), kontroler direktno kreira i šalje `EmailNotifikacija` objekt. Ovaj pristup uzrokuje sljedeće probleme:

- Svaki kontroler koji treba poslati notifikaciju mora direktno znati za `EmailNotifikacija` klasu i logiku njenog kreiranja.
- Dodavanje novog tipa notifikacije (npr. push notifikacija, SMS) zahtijeva izmjenu svakog kontrolera posebno.
- Nema uniformnog mjesta koje prati sve događaje u sistemu.

3.1.2 Rješenje primjenom Observer paterna

Uvodi se interfejs `IDogadjajObserver` koji definira metodu `onDogadjaj(tip, podaci)`. Klase koje trebaju reagovati na događaje implementiraju ovaj interfejs. Uvodi se klasa `DogadjajPublisher` koja upravlja listom observera i poziva ih kada se dogodi relevantna akcija.

IDogadjajObserver «interface»

- `+onDogadjaj(tip: TipDogadjaja, podaci: object) : void`

DogadjajPublisher

Atributi: `-observeri : List<IDogadjajObserver>`

Metode:

- `+subscribe(observer: IDogadjajObserver) : void`
- `+unsubscribe(observer: IDogadjajObserver) : void`
- `+notify(tip: TipDogadjaja, podaci: object) : void`

EmailNotifikacijaObserver implementira `IDogadjajObserver`

Atributi: `-emailServis : IEmailServis`

Reaguje na događaje kreiranjem i slanjem odgovarajuće `EmailNotifikacija`.

LogObserver implementira `IDogadjajObserver`

Atributi: `-log : Log`

Reaguje na svaki događaj upisivanjem zapisa u sistemski log.

TipDogadjaja «enumeration»

`PRIJAVA_NA_OGLAS, SLANJE_PONUDE, VERIFIKACIJA_ODOBRENA, VERIFIKACIJA_ODBIJENA,`

NOVA_REGISTRACIJA

Prednosti:

- Kontroleri više ne znaju ništa o notifikacijama — samo objavljuju događaj kroz `DogadjajPublisher`.
- Novi tip observera (npr. SMS notifikacije) dodaje se bez izmjene postojećeg koda.
- `LogObserver` automatski prati sve događaje bez izmjene poslovne logike.

3.2 Strategy patern — Algoritam rangiranja studenta

3.2.1 Problem bez paterna

Trenutno klasa `StudentProfil` koristi pondere iz klase `RangParametri` (prosjek ocjena, ECTS bodovi, broj vještina, broj projekata) i izračunava rang po jedinstvenoj formuli. Ovaj pristup ima sljedeće nedostatke:

- Nije moguće lako uvesti alternativnu formulu rangiranja bez izmjene klase `StudentProfil`.
- Različite firme ili scenariji mogu zahtijevati drugačiji pristup rangiranju (npr. firma iz IT sektora može više vrednovati tehničke vještine nego prosjek ocjena).
- Testiranje algoritma je otežano jer je logika direktno ugrađena u klasu profila.

3.2.2 Rješenje primjenom Strategy paterna

Uvodi se interfejs `IRangStrategija` koji definira metodu `izracunajRang()`. Klasa `StudentProfil` više ne izračunava rang direktno, već delegira tu operaciju konkretnoj strategiji koja joj je proslijeđena.

IRangStrategija «interface»

- `+izracunajRang(profil: StudentProfil, parametri: RangParametri): double`

StandardnaRangStrategija implementira `IRangStrategija`

Kombinuje sve faktore (prosjek, ECTS, vještine, projekti) koristeći pondere iz `RangParametri`. Ovo je podrazumijevana strategija sistema.

AkademskiRangStrategija implementira `IRangStrategija`

Stavlja veći naglasak na akademski uspjeh (prosjek ocjena i ECTS bodove), zanemarujući projekte i vještine. Pogodna za firme koje traže visok akademski uspjeh.

PrakticniRangStrategija implementira `IRangStrategija`

Stavlja veći naglasak na praktično iskustvo (vještine i projekti), s manjim ponderom za akademski uspjeh. Pogodna za IT firme koje traže praktično iskustvo.

Izmjena u klasi StudentProfil:

- Dodaje se atribut `-rangStrategija : IRangStrategija`
- Metoda `izracunajRang()` delegira poziv:
`return rangStrategija.izracunajRang(this, parametri);`

Prednosti:

- Nove strategije rangiranja dodaju se bez izmjene klase `StudentProfil`.
- Strategija se može mijenjati u runtime-u — svaka firma može koristiti strategiju koja odgovara njenom profilu.
- Svaka strategija može biti nezavisno testirana.
- Poštuje *Open/Closed Principle* — sistem je otvoren za proširenje novim strategijama, ali zatvoren za izmjenu postojećeg koda.

4 Pregled dodanih klasa

Klasa / Interfejs	Patern	Uloga
IDogadjajObserver	Observer	Interfejs za sve observere u sistemu
DogadjajPublisher	Observer	Centralni publisher koji obavještava observere
EmailNotifikacijaObserver	Observer	Reaguje na događaje slanjem email notifikacija
LogObserver	Observer	Reaguje na događaje upisivanjem u sistemski log
TipDogadjaja	Observer	Enumeracija tipova događaja u sistemu
IRangStrategija	Strategy	Interfejs za sve strategije rangiranja
StandardnaRangStrategija	Strategy	Podrazumijevana strategija s ponderisanom formulom
AkademskiRangStrategija	Strategy	Strategija s naglaskom na akademski uspjeh
PrakticniRangStrategija	Strategy	Strategija s naglaskom na praktično iskustvo

Tablica 1: Pregled novih klasa uvedenih primjenom paterna ponašanja